서울시립대학교 · 전자전기컴퓨터설계실험II

LAB5 실험 후 보고서

FPGA5 / MCU1 — 피아노 · 신호등 제어 · ADC · STM32 인터럽트와 PWM (제공 예제 3개 · 과제3 · 과제4)

이름
형종희

학번
2023440145

조
C조

실험일
2026-10-19

작성일
2026-10-19

FPGA 보드
Combo II-DLD S75 · xc7s75fgga484-1

MCU 보드
NUCLEO-G474RE

도구
Vivado 2026.1 · Icarus Verilog 12.0 · STM32Cube (GCC 14.3.1)

📦 GitHub 제출 정보

항목	값
저장소	https://github.com/hyeongjonghee/electronics2-LABs (Public)
제출 태그	`lab5-submit` — 이 보고서(HTML · PDF)와 시연 영상이 들어간 최종 커밋을 가리킴
이전 커밋	실험 전 보고서 태그 `lab5-prelab` (`f3db66c`) · 설계 소스와 시뮬레이션 `569be96` · 보드 XDC · 보드용 최상위 · 영상 `243b2ea`
LAB5 경로	`LAB5/circuits/LAB5-01~03/` (src · sim · constraints · results), `LAB5/stm32/ex1~3 · task3~4/` , `LAB5/evidence/videos/`
STM32 기반 프로젝트	`Glaysia/ece2_474` v1.1.0 = `dd94f01` . 학생이 바꾼 파일은 `Core/Src/ece2_474_routine.c` 하나

0. 결과 요약

10/19 실습에서 FPGA 개별 실습 3개와 STM32_1의 제공 예제 3개 · 과제 2개를 모두 보드에서 동작시켰다. 과제3과 과제4(디버거 사용 포함)는 조교 확인을 받았다. FPGA 핀은 실험 당일 교재 핀표와 신호 이름 단위로 대조해 XDC를 확정했다. ADC는 처음에 LED가 모두 켜진 채(0xFF) 바뀌지 않았고, 보드용 최상위 모듈의 직렬 타이밍을 고치고 핀 배치를 ADC 칩 기준으로 다시 잡은 뒤 동작했다(3장 ⑧).

#	실험	시뮬레이션	교재 핀 대조	자원 · 타이밍 (보드 빌드)	보드 결과
1	피아노 (FPGA)	XSim · Icarus PASS	p.396 · rst 만 바꿈	LUT 646 · FF 43 · IOB 11 · WNS +14.49 ns	**8음 · 무음 · 리셋 ✔**
2	신호등 제어 (FPGA)	XSim · Icarus PASS	p.657~658 · LED 24개	LUT 32 · FF 32 · IOB 27 · WNS +14.97 ns	**순서 · 점멸 ✔**
3	ADC (FPGA)	XSim · Icarus PASS (수업 TB + 보드판 TB)	3.4절 · 핀 재배치	LUT 29 · FF 49 · IOB 17 · WNS +16.06 ns	**수정 후 ✔**
4	STM32_1 제공 예제 3개	호스트 모의 실행	—	—	**토글 · 1 s · 998 Hz ✔**
5	STM32_1 과제3	호스트 모의 실행	—	FLASH 25,976 B	**조교 확인 ✔**
6	STM32_1 과제4 · 디버거	호스트 모의 실행	—	FLASH 26,308 B	**조교 확인 ✔**

실험 전 보고서 대비 바뀐 점

- **피아노**: 리셋을 교재 p.396대로 DIP SW1(Y1)에 연결했다(실험 전 계획은 키패드 *). 키 8개와 피에조는 계획과 같았다.
- **신호등**: 교재 p.657~658의 신호등 모듈 LED 24개를 모두 연결하는 보드용 최상위 `traffic_board_top`을 만들었다. 점멸 입력은 교재대로 키패드 2(N8)이다.
- **ADC**: 수업 모듈을 그대로 쓰지 않고 보드용 최상위 `adc_board_top`에서 직렬 통신을 다시 만들었다. DIN을 SCLK 상승 에지에서 바꾸고 프레임 사이 CS HIGH 시간을 1 µs로 늘렸다. 핀은 교재 표의 하드웨어 이름(ADC 칩 기준)에 맞췄다. AD7908 모델을 넣은 테스트벤치를 새로 만들어 XSim으로 검증했다.
- **교재 표기 오류**: ADC · DAC 핀 설정표의 `clk = N8 (Main Clock)`은 키패드 2번 핀이다. 신호등 표와 LAB3 · 4에서 확인한 메인 클록 B6을 사용했다.

목차

1. 실험 1 · 피아노
2. 실험 2 · 신호등 제어
3. 실험 3 · ADC
4. 실험 4 · STM32_1 제공 예제
5. 실험 5 · 과제3
6. 실험 6 · 과제4
7. 고찰
8. 참고문헌

1. 실험 1 · 피아노 (piano)

① 목적

키패드 8개 중 하나를 누르는 동안 도(262 Hz)~높은 도(523 Hz)의 사각파를 피에조로 내보낸다. 50 MHz 클록을 세어 음의 주파수를 만드는 분주 방법을 보드에서 확인한다.

② 이론

출력을 반주기마다 뒤집으면 사각파가 된다. 반주기를 클록 칸 수로 바꾸면 다음과 같다.

$$N = \left\lfloor \frac{f_{clk}}{2f} \right\rfloor, \qquad f_{\text{실제}} = \frac{f_{clk}}{2N} \quad (\text{라 } 440\ \text{Hz}:\ N = 56{,}818)$$

정수로 자른 오차는 8음 모두 0.02 cent 이하로 들리지 않는 수준이다(실험 전 보고서 1장). `note_select`는 키가 정확히 하나일 때만 음을 고르고, 0개나 2개 이상이면 주파수 0(무음)을 낸다.

③ 포트 · 보드 매핑과 교재 대조 (교재 p.396)

포트	교재 핀 · 장치	XDC	대조
clk_50mhz	B6 · Main_clock	B6	일치
rst_p	Y1 · DIPSW_1	Y1	계획(L7)에서 변경
key[7] ~ key[4]	K4 · N8 · N4 · N1 (SW_1~4)	K4 · N8 · N4 · N1	일치
key[3] ~ key[0]	P6 · N6 · L5 · J2 (SW_5~8)	P6 · N6 · L5 · J2	일치
piezo	Y21 · piezo	Y21	일치

교재의 SW_1~8은 키패드 1~8번이다. rst_p는 HIGH 활성이므로 **DIP SW1이 ON이면 리셋(무음)**, OFF여야 연주된다.

④ XDC

```
# LAB5-01 piano (기본 핀: 실험 당일 교재와 다르면 이 줄만 수정)
set_property PACKAGE_PIN B6  [get_ports clk_50mhz]
```

```
set_property PACKAGE_PIN Y1  [get_ports rst_p]
set_property PACKAGE_PIN K4  [get_ports {key[7]}]
set_property PACKAGE_PIN N8  [get_ports {key[6]}]
set_property PACKAGE_PIN N4  [get_ports {key[5]}]
set_property PACKAGE_PIN N1  [get_ports {key[4]}]
set_property PACKAGE_PIN P6  [get_ports {key[3]}]
set_property PACKAGE_PIN N6  [get_ports {key[2]}]
set_property PACKAGE_PIN L5  [get_ports {key[1]}]
set_property PACKAGE_PIN J2  [get_ports {key[0]}]
set_property PACKAGE_PIN Y21 [get_ports piezo]
set_property IOSTANDARD LVCMOS33 [get_ports *]
create_clock -name clk_50mhz -period 20.000 [get_ports clk_50mhz]
set_false_path -from [get_ports {rst_p key[*]}]
```

RTL(piano · note_select · square_tone)은 수업 코드 그대로이며 실험 전 보고서 1장 ④와 저장소 `LAB5/circuits/LAB5-01/src/` 에 있다.

⑤ Vivado 시뮬레이션

수업 TB(`CLK_HZ=10000` 로 축소)를 XSim으로 실행해 `PASS piano: all 8 tones, silence, multiple keys, reset` 을 확인했다. 8음의 반주기 측정값(190 · 170 · 150 · 140 · 120 · 110 · 100 · 90 ns)은 계산값과 모두 같았다.

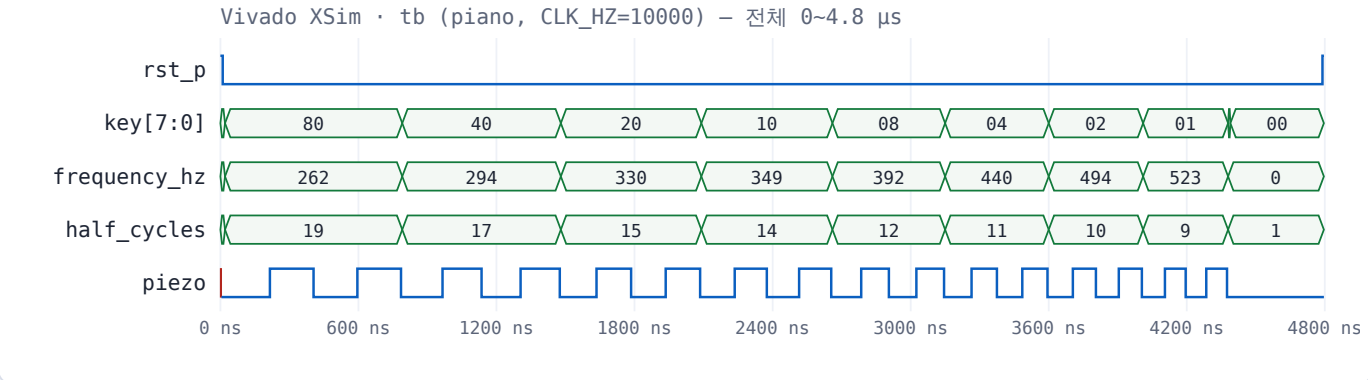


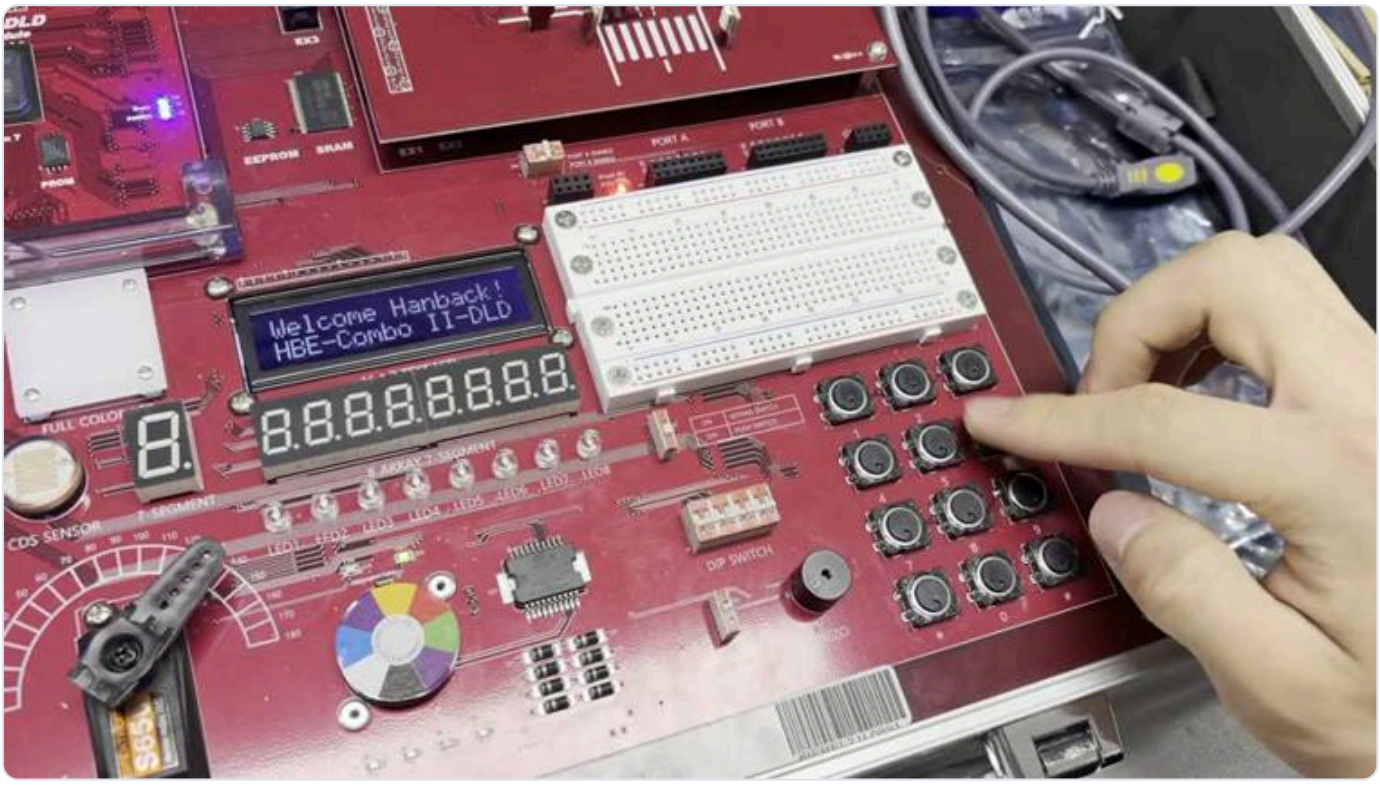
그림 1-1. Vivado XSim 파형. 키가 도(80)→높은 도(01)로 바뀔 때마다 반주기가 19→9칸으로 줄어 piezo 반전이 빨라진다. C0(두 키)과 00(무음)에서는 piezo = 0.

⑥ 합성 · 구현 결과

LUT 646 · FF 43 · IOB 11 · DSP 0. 50 MHz 제약에서 WNS +14.490 ns, WHS +0.265 ns로 타이밍을 만족했다. LUT가 많은 것은 반주기를 구하는 변수 나눗셈 때문이다(고찰 2).

⑦ 보드 검증

확인 항목	기대	결과
키패드 1→8	도 · 레 · 미 · 파 · 솔 · 라 · 시 · 높은 도 순서로 음이 올라감	일치
두 키 동시 입력	무음	일치
DIP SW1 ON	리셋 → 무음	일치



피아노 시연 · 키패드를 누르는 동안 피에조에서 음이 남 (소리는 영상에서 확인)

영상: LAB5/evidence/videos/LAB5-01_piano.mp4 (GitHub에서 열기 ↗)

2. 실험 2 · 신호등 제어 (traffic)

① 목적

네 방향 교차로의 차량 신호(빨강 · 노랑 · 초록 · 좌회전)와 보행자 신호(빨강 · 초록)를 1초 단위로 진행시키고, 보행자 초록 점멸과 전체 점멸 모드를 보드의 신호등 모듈에서 확인한다.

② 이론

새 1 Hz 클록을 만들지 않고, 50 MHz로 세다가 1초에 한 클록만 1이 되는 `tick`으로 순서를 진행한다. 한 방향은 13초(초록 0~10초 · 노랑 11~12초), 한 바퀴는 $4 \times 13 = 52$초이다. 보행자 초록은 0~3초 켜짐, 4~10초는 1 Hz 점멸이다. `flicker=1`이면 노랑 4개와 보행자 빨강 4개만 함께 점멸한다.

③ 포트 · 보드 매핑과 교재 대조 (교재 p.657~658)

교재 표의 인덱스는 **[3] = 남(S), [2] = 북(N), [1] = 서(W), [0] = 동(E)**이다. 따라서 방향 0~3은 남 → 북 → 서 → 동 순서가 된다.

포트	[3] 남	[2] 북	[1] 서	[0] 동	대조
led_red	B13	G14	C17	C12	일치
led_yellow	L18	A14	C18	D13	일치
led_green	E16	C16	A20	E11	일치
led_left (교재 led_left_o)	F15	D15	C19	F10	일치
led_walk_red	A19	G13	E18	C11	일치
led_walk_green	E19	F12	A16	G11	일치

포트	교재	XDC	대조
clk_50mhz	B6 · Main Clock	B6	일치
rst_p	K4 · SW_1 (키패드 1)	K4	일치
flicker	N8 · SW_2 (키패드 2)	N8	일치
sw_flicker	N4 · SW_3	사용 안 함	수업 코드에 없음

수업 코드는 원본의 수동 점멸 토글(`sw_flicker`)을 빼고, `flicker` 가 HIGH인 동안 점멸하도록 정리했다. 그래서 키패드 2를 누르고 있는 동안만 점멸 모드가 된다.

④ 보드용 최상위 · XDC

```
`timescale 1ns/1ps
// 보드 연결 (교재 3.5절 p.657~658): [3]=S, [2]=N, [1]=W, [0]=E. rst = 키패드1(K4), flicker = 키패드2(N8, 누르는
// LED가 반대로 켜지면 ACTIVE_LOW를 1로 바꿔 재빌드한다.
module traffic_board_top(
    input wire clk_50mhz, rst_p, flicker,
    output wire [3:0] led_red, led_yellow, led_green, led_left, led_walk_red, led_walk_green
);
    localparam ACTIVE_LOW = 1'b0;
    wire [3:0] r, y, g, l, wr, wg;
    traffic u_traffic(clk_50mhz, rst_p, flicker, r, y, g, l, wr, wg);
    assign led_red        = r  ^ {4{ACTIVE_LOW}};
    assign led_yellow     = y  ^ {4{ACTIVE_LOW}};
    assign led_green      = g  ^ {4{ACTIVE_LOW}};
    assign led_left       = l  ^ {4{ACTIVE_LOW}};
    assign led_walk_red   = wr ^ {4{ACTIVE_LOW}};
    assign led_walk_green = wg ^ {4{ACTIVE_LOW}};
endmodule
```

XDC: `LAB5/circuits/LAB5-02/constraints/lab5_traffic.xdc` (위 표의 27개 핀, LVCMOS33).

⑤ Vivado 시뮬레이션

수업 TB(`CLK_HZ=4` , 1초 = 4클록)가 4방향 × 13단계 × 4클록 동안 매 클록 LED 24개를 비교했고 `PASS traffic: full cycle, pedestrian blink, flicker, reset` 이었다. `traffic_board_top` 은 출력을 그대로 연결하는 모듈이라(`ACTIVE_LOW = 0`) 따로 검증하지 않았다.

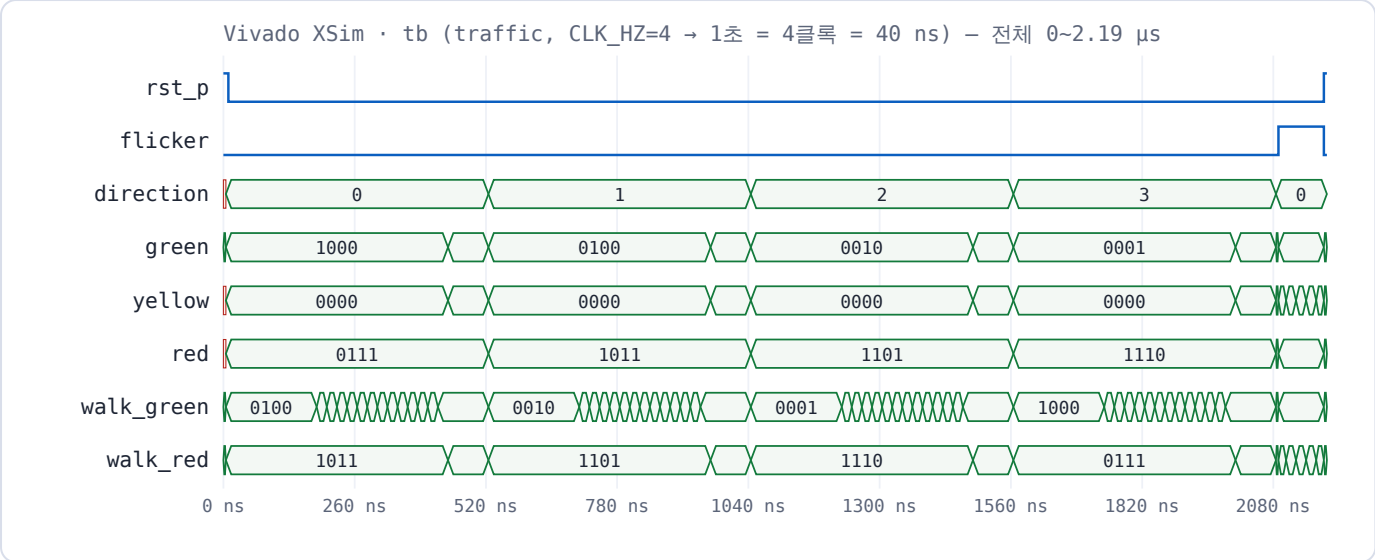


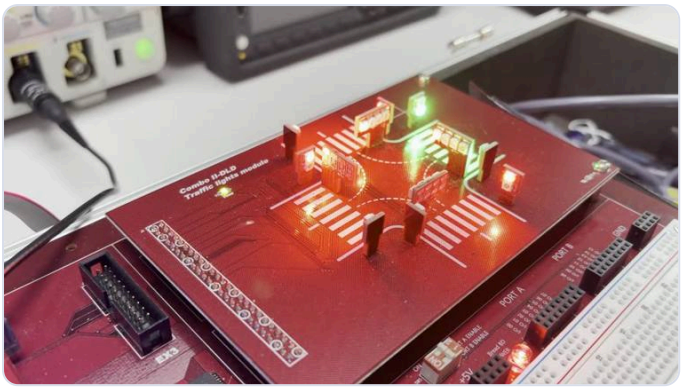
그림 2-1. Vivado XSim 파형. direction 0→3 동안 green의 1이 1000(남)→0100(북)→0010(서)→0001(동)으로 옮겨 가고, 각 방향 끝 2초는 yellow. 끝의 flicker 구간에서 yellow와 walk_red가 0000/1111로 함께 점멸한다.

⑥ 합성 · 구현 결과

LUT 32 · FF 32 · IOB 27(클록 1 + 입력 2 + LED 24). WNS +14.967 ns, WHS +0.244 ns.

⑦ 보드 검증

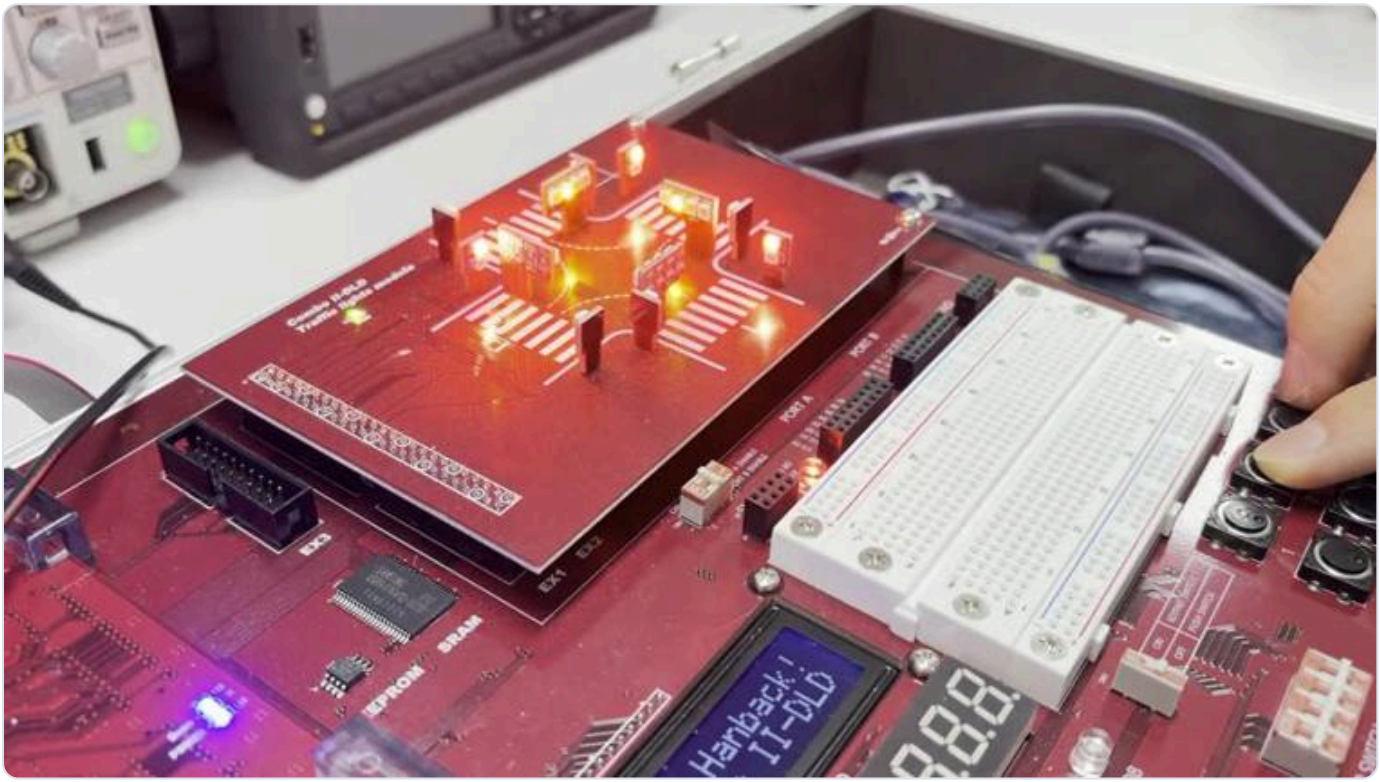
확인 항목	기대	결과
방향 순서	남 → 북 → 서 → 동, 현재 방향만 초록 + 좌회전, 나머지 빨강	일치
노랑	각 방향 마지막 2초	일치
보행자 초록	4초 켜짐 → 7초 점멸 → 꺼짐	일치
키패드 2 누름	노랑 · 보행자 빨강만 1 Hz 점멸, 떼면 남쪽부터 다시 시작	일치
LED 극성	출력 1 = 켜짐 (반전 필요 없음)	일치



한 방향 초록 + 좌회전, 나머지 차량 빨강



다음 방향으로 넘어간 뒤의 초록



키패드 2를 누르는 동안의 점멸 모드 (노랑 · 보행자 빨강)

영상: LAB5/evidence/videos/LAB5-02_traffic.mp4 (GitHub에서 열기 ↗)

3. 실험 3 · ADC (adc)

① 목적

FPGA가 보드의 ADC 칩(AD7908, 8채널 · 8비트)에 직렬 명령을 보내고 변환값을 받아 LED1~8에 표시한다. 교재에 따르면 ch0에는 CdS 조도 센서, ch1에는 가변저항이 연결되어 있다. 채널은 DIP SW1~3으로 고른다.

② 이론 · AD7908 직렬 인터페이스 [3]

- CS가 내려가면 DOUT에 첫 비트(앞 0)가 나오고, 이후 **SCLK 하강 에지마다** DOUT이 다음 비트로 바뀐다. 출력 16비트는 {0, 주소 3비트, 데이터 8비트, 0000}이다.
- DIN은 **SCLK 하강 에지에서** 읽는다. 처음 12비트가 제어 레지스터 {WRITE, SEQ, -, ADD2~0, PM1~0, SHADOW, -, RANGE, CODING}이다.
- 변환 결과는 **직전 프레임에서 지정한 채널**의 값이다. 채널을 바꾸면 한 프레임 늦게 반영된다.
- CS 상승부터 다음 변환까지 **최소 50 ns**(t_{QUIET})를 쉬어야 한다.

수업 명령 워드 `{101, ch, 11, 0, 111, 0000}`은 WRITE = 1, 주소 = ch, 정상 전력(PM = 11), RANGE = 1(0 V~REF_{IN}), CODING = 1(부호 없는 이진수)이다. SCLK는 50 MHz/(2×25) = 1 MHz이다.

③ 포트 · 보드 매핑과 교재 대조 (교재 3.4절 실습 36)

포트	교재 포트표	교재 하드웨어 이름	최종 XDC	판단
clk_50mhz	N8 · Main Clock	Main Clock	B6	**교재 오기 (N8 = 키패드 2)**
rst_p	K4	SW_1 (키패드 1)	K4	**일치**
add_sel[2:0]	Y1 · W3 · U2	DIP_SW1~3	Y1 · W3 · U2	**일치**
adc_din (FPGA → ADC)	H3	SPI_AD	**H5** (SPI_DIN)	**ADC 칩 기준 이름으로 재배치**
adc_dout (ADC → FPGA)	H5	SPI_DIN	**J6** (SPI_DOUT)	**재배치**
adc_ncs	J6	SPI_DOUT	**H3** (SPI_AD)	**재배치**
adc_sclk	H7	SPI_SCK	H7	**일치**
led[7:0]	L4 · M4 · M2 · N7 · M7 · M3 · M1 · N5	LED1~8	같음	**일치**

교재의 하드웨어 이름 열은 ADC 칩 쪽에서 본 이름이다. 칩의 DIN 핀(SPI_DIN)에는 FPGA의 `adc_din`을, 칩의 DOUT(SPI_DOUT)에는 `adc_dout`을, SPI_AD에는 칩 선택(CS)을 연결해야 동작했다. 교재 포트표는 이 세 핀이 한 칸씩 어긋나 있었다(⑧).

④ 보드용 최상위 (adc_board_top)

수업 모듈 `adc_command`로 명령 워드를 만들고, 직렬 송수신은 새로 작성했다. 수업 `serial_frame16`과 다른 점은 주석 1)~3)이다.

```
`timescale 1ns/1ps
// 보드 시연용 ADC(AD7908) 연결. 수업 serial_frame16과 다른 점:
//  1) adc_din은 SCLK 상승 에지에서 바꿔 ADC가 읽는 하강 에지에서 안정되게 한다.
//  2) 프레임 사이 CS HIGH 시간 1 us (t_QUIET 50 ns 이상).
//  3) DOUT은 SCLK 상승 에지에서 읽고 DB7~DB0을 LED1~8에 표시한다.
```

```
module adc_board_top #(parameter HALF_CYCLES=25, GAP_CYCLES=50)(
    input  wire clk_50mhz, rst_p,
    input  wire [2:0] add_sel,
    output reg  adc_sclk, adc_ncs, adc_din,
    input  wire adc_dout,
    output reg  [7:0] led
);
    wire [15:0] command;
    adc_command u_command(add_sel, command);
    reg [15:0] tx, rx;
    reg [7:0] count;
    reg [4:0] edges;
    reg active, dout_s;
    always @(posedge clk_50mhz) dout_s <= adc_dout;
    always @(posedge clk_50mhz) begin
        if (rst_p) begin
            adc_sclk <= 1'b1; adc_ncs <= 1'b1; adc_din <= 1'b1;
            count <= 0; edges <= 0; active <= 1'b0;
            tx <= 0; rx <= 0; led <= 0;
        end else if (!active) begin
            adc_sclk <= 1'b1;
            if (count == GAP_CYCLES-1) begin
                count <= 0; edges <= 0; active <= 1'b1;
                adc_ncs <= 1'b0; tx <= command; adc_din <= command[15];
            end else count <= count + 1'b1;
        end else if (count == HALF_CYCLES-1) begin
            count <= 0;
            if (adc_sclk) adc_sclk <= 1'b0;
            else begin
                adc_sclk <= 1'b1;
                rx <= {rx[14:0], dout_s};
                adc_din <= tx[14];
                tx <= {tx[14:0], 1'b0};
                if (edges == 5'd15) begin
                    active <= 1'b0; adc_ncs <= 1'b1;
                    led <= rx[11:4];
                end
                edges <= edges + 1'b1;
            end
        end else count <= count + 1'b1;
    end
endmodule
```

⑤ Vivado 시뮬레이션

(a) 수업 TB. `adc` 모듈(HALF_CYCLES = 2)로 프레임 3개를 검사해 `PASS ADC: 3 serial frames, command bits, sample, reset` 이었다(실험 전 보고서 3장).

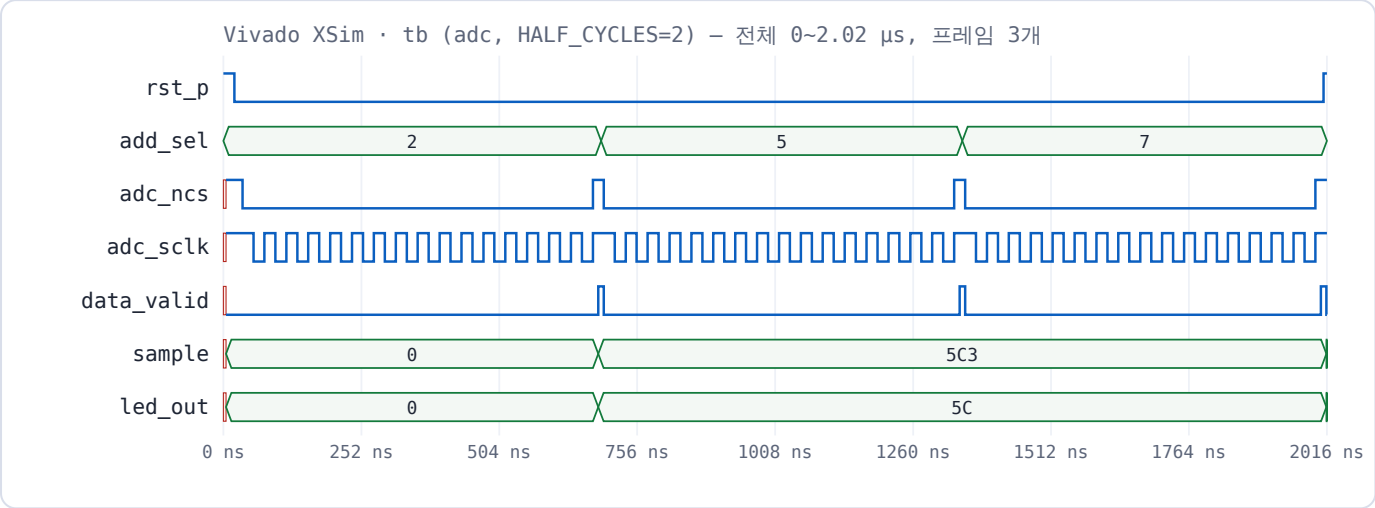


그림 3-1. 수업 TB의 Vivado XSim 파형. 프레임마다 SCLK 16펄스, 끝에서 data_valid가 한 클록 1이 되고 sample = 5C3.

(b) 보드판 TB (새로 작성). `sim/ad7908_model.sv` 는 ②의 칩 동작을 그대로 흉내 내고, DIN이 SCLK 하강 에지와 **같은 순간**에 바뀐 횟수와 CS HIGH 시간을 잰다. `sim/tb_board.sv` 는 실제 50 MHz · 1 MHz 조건에서 `adc_board_top` 과 수업 `adc` 를 각각 모델에 연결해 비교한다. ch0 = 0x3C, ch1 = 0x5A로 시작하고, 가변저항을 돌린 것처럼 ch1을 0xA7로 바꾼 뒤 채널을 0으로 바꿨다.

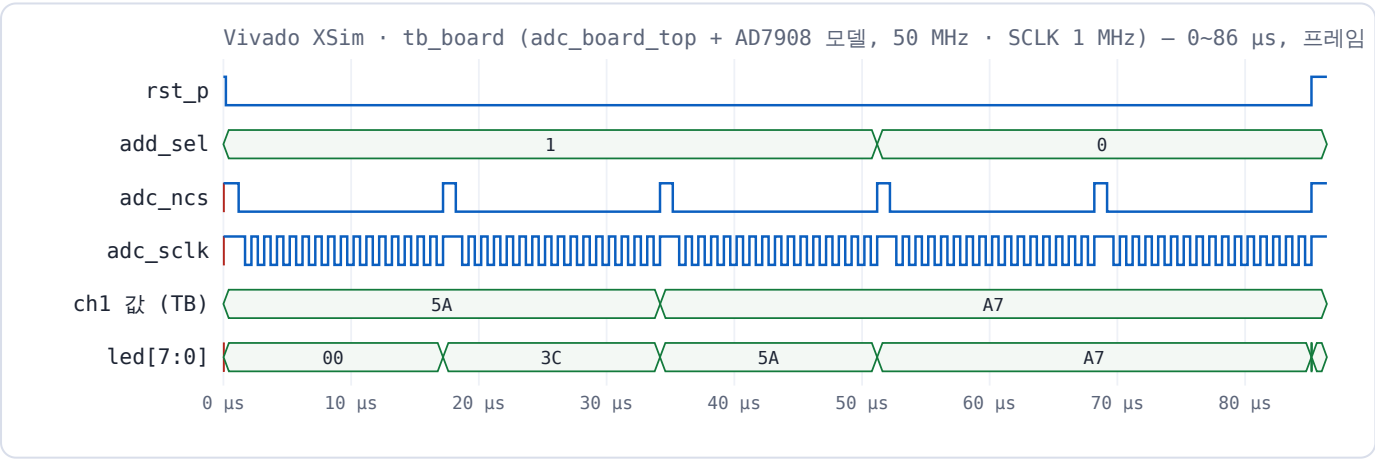


그림 3-2. 보드판 TB의 Vivado XSim 파형. LED가 3C(전원 직후 주소 0) → 5A → A7(값 변경) → A7(채널 변경 직후, 한 프레임 지연) → 3C로 바뀐다.

프레임	보낸 명령	변환 채널	LED	판정
1	A770 (ch1 지정)	ch0 (전원 직후 주소 0)	3C	일치
2	A770	ch1	5A	일치
3	A770	ch1 (값 A7로 바꿈)	A7	일치
4	A370 (ch0 지정)	ch1 (한 프레임 지연)	A7	일치
5	A370	ch0	3C	일치

Icarus와 XSim 모두 같은 결과와 `PASS adc_board: 5 frames, command bits, channel latency, LED value, reset`을 출력했다. 같은 모델에 붙인 두 설계의 타이밍 측정값은 다음과 같다.

측정 (모델이 잰 값)	adc_board_top (수정판)	수업 adc	AD7908 요구
SCLK 하강 에지와 같은 순간에 DIN이 바뀐 횟수	**0**	**38회**	하강 에지 전후로 DIN 유지 (setup · hold)
DIN이 바뀌는 시점	SCLK 상승 에지 (하강 에지 500 ns 전)	SCLK 하강 에지와 같은 클록	DIN은 하강 에지에서 읽음
프레임 사이 CS HIGH	**1000 ns**	**40 ns**	t_{QUIET} ≥ 50 ns
A7을 변환했을 때 LED	A7	4E (= A7을 왼쪽으로 1비트 민 값)	—

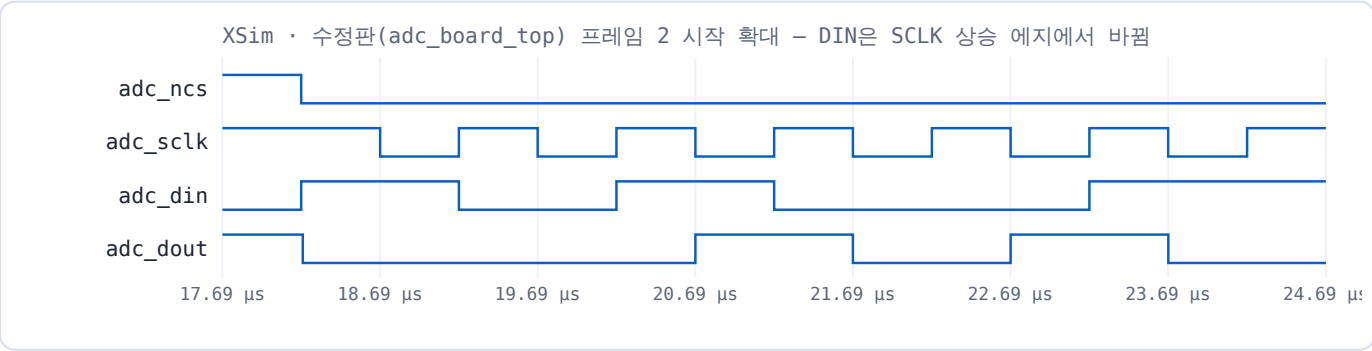


그림 3-3. 수정판: DIN(adc_din)은 SCLK가 올라갈 때 바뀌어, ADC가 읽는 하강 에지에서는 반주기(500 ns) 동안 안정되어 있다.

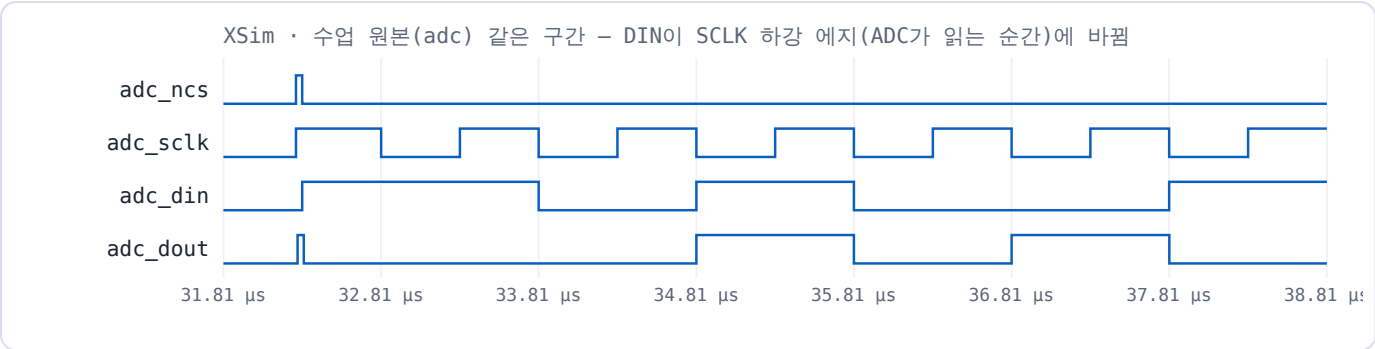


그림 3-4. 수업 원본: DIN이 SCLK 하강 에지와 같은 클록에 바뀐다. 실제 칩에서는 바뀌기 전 값과 바뀐 값 중 무엇을 읽을지 보장되지 않는다.

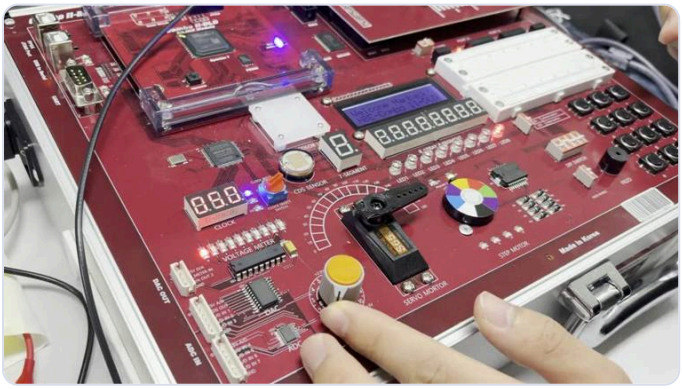
수업 원본은 SCLK 상승 에지 16번에서 DOUT을 읽은 뒤 하위 12비트를 sample로 쓴다. CS 하강에서 나오는 앞 0을 세지 않으므로, AD7908 출력 형식에서는 데이터가 한 비트 밀린다(A7 → 4E). 수정판은 16번째 상승 에지 직전의 `rx[11:4]`를 읽어 DB7~DB0을 그대로 표시한다.

⑥ 합성 · 구현 결과

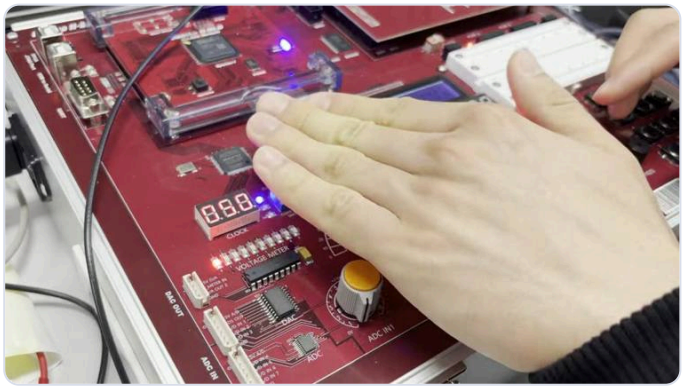
LUT 29 · FF 49 · IOB 17(클록 1 + 리셋 1 + 채널 3 + 직렬선 4 + LED 8). WNS +16.060 ns, WHS +0.140 ns.

⑦ 보드 검증

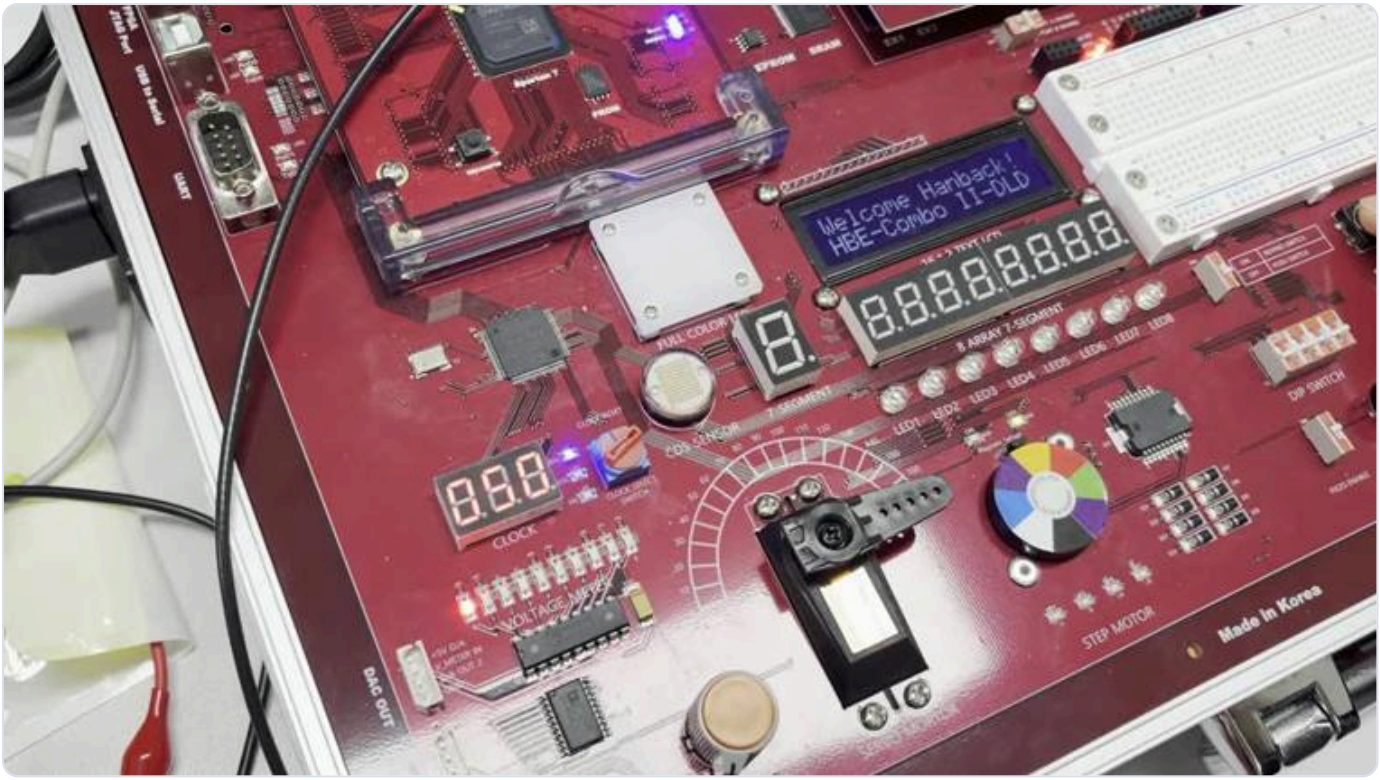
확인 항목	설정	기대	결과
가변저항	DIP SW3만 ON (ch1)	돌리는 방향에 따라 LED 값이 커지고 작아짐	일치
CdS 센서	SW1~3 모두 OFF (ch0)	손으로 가리면 값이 바뀜	일치
리셋	키패드 1 누름	LED 모두 꺼짐	일치



ch1 · 가변저항을 돌리며 LED1~8 값 변화 확인



ch0 · CdS 센서를 손으로 가림



ADC 동작 화면 (LED1~8 = 변환값 상위 8비트)

영상: LAB5/evidence/videos/LAB5-03_adc.mp4 (GitHub에서 열기 ↗)

⑧ 문제 해결 과정

처음 비트스트림부터 LED가 **모두 켜진 채(0xFF)** 가변저항을 돌려도 바뀌지 않았다. 키패드 1(리셋)을 누르면 꺼졌으므로 FPGA와 LED 경로는 정상이었다. ADC 쪽 점퍼나 활성화 스위치는 없었다. 시도한 순서는 다음과 같다.

#	직렬 통신	din · dout · ncs 핀	결과
1	수업 serial_frame16	H3 · H5 · J6 (교재 포트표)	**LED 모두 켜짐**
2	수업 serial_frame16	J6 · H5 · H3 (din · ncs 맞바꿈)	**LED 모두 켜짐**
3	수정판 adc_board_top	H3 · H5 · J6 (교재 포트표)	**LED 모두 켜짐**
4	수정판 adc_board_top	**H5 · J6 · H3** (하드웨어 이름 기준)	**동작**

칩 선택이 칩에 닿지 않으면 칩이 DOUT을 내보내지 않는다. 그러면 FPGA는 계속 1을 읽어 0xFF가 되는데, 1~3번의 증상이 이것과 같다. 4번에서 핀을 하드웨어 이름대로 맞추자 동작했으므로, **직접 원인은 핀 배치**였다. 수업 원본 타이밍을 4번 핀으로 시험하지는 않아서 타이밍 수정이 꼭 필요했는지는 보드로 확인하지 못했다. 다만 ⑤(b)의 모델 비교에서 수업 원본은 DIN을 칩이 읽는 순간에 바꾸고(38회), CS HIGH가 40 ns로 데이터시트 최솟값보다 짧다. 데이터 정렬도 한 비트 어긋나므로, 수정판을 쓰는 것이 맞다고 판단했다.

4. 실험 4 · STM32_1 제공 예제 3개

① 목적

메인 루프 대신 인터럽트 함수에서 일을 처리하는 구조(버튼 EXTI, TIM6 주기 인터럽트)와 TIM3 PWM 출력을 예제로 먼저 확인한다.

② 이론

$$f_{TIM6} = \frac{100\ \text{MHz}}{(9999+1)(999+1)} = 10\ \text{Hz}, \qquad f_{PWM} = \frac{100\ \text{MHz}}{(99+1)(999+1)} = 1\ \text{kHz}, \qquad D$$

③ 코드

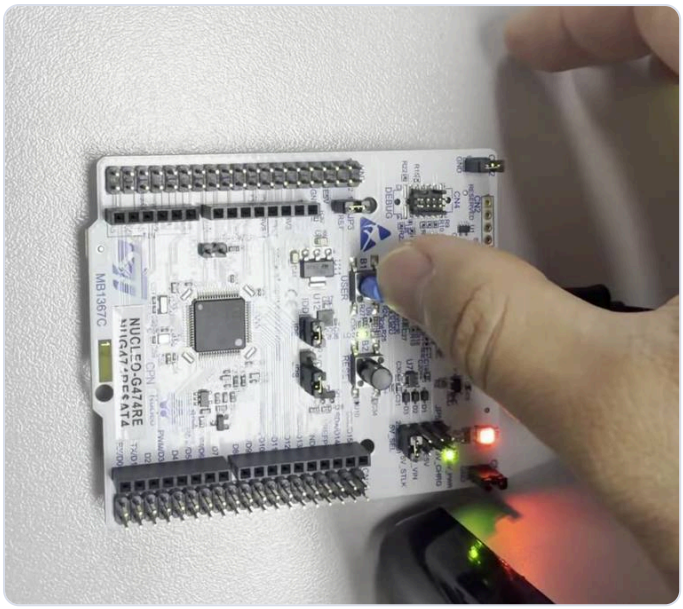
예제마다 수업 템플릿(dd94f01)에 매뉴얼의 코드만 넣었다. 저장소 `LAB5/stm32/ex1~3/` 에 있다. ① `button_isr_p7` 에서 200 ms 떨림 검사 후 LD2 토글, ② `tim6_isr_p4` 에서 10회마다 LD2 토글, ③ `ece2_474_init` 에서 PWM 시작과 비교값 500.

④ 모의 실행 (실험 전)

같은 코드를 PC에서 가짜 HAL과 함께 실행해, ①은 떨림 2회를 무시하고 버튼 3번만 인정, ②는 버튼과 관계없이 1 s마다 반전, ③은 비교값 500 유지를 확인했다.

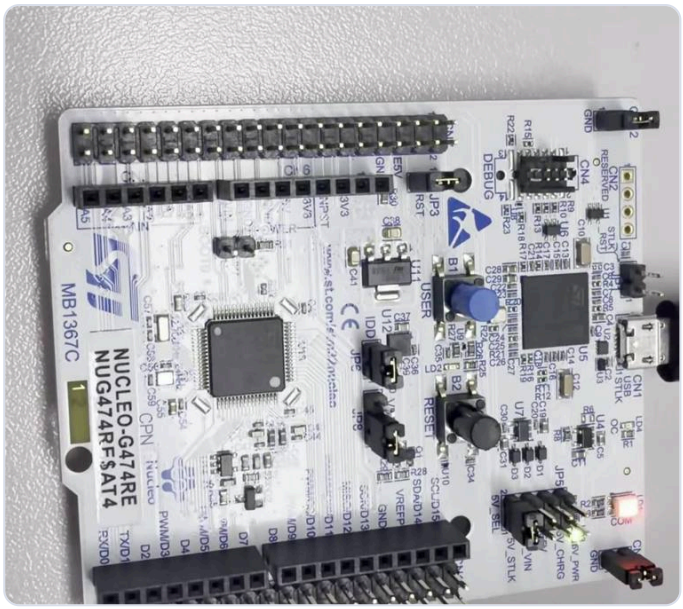
⑤ 보드 검증

예제	기대	측정 · 관찰	결과
① 버튼 ISR	버튼마다 LD2 한 번 토글	토글 확인, Serial에 Hello, World! (COM5)	일치
② TIM6	LD2 1 s 켜짐 / 1 s 꺼짐	버튼을 눌러도 리듬 유지	일치
③ PWM 50 %	1 kHz · HIGH 0.5 ms · 0~3.3 V	998.4 Hz(측정값), 트리거 표시 999 Hz, 최대 3.44 V · 최소 −0.08 V, HIGH와 LOW 길이 같음	일치



예제 ① 버튼을 눌러 LD2 토글

영상: LAB5/evidence/videos/LAB5-04_ex1_button.mp4 (GitHub에서 열기 ↗)



예제 ② TIM6 1 s 점멸 (LD2)

영상: LAB5/evidence/videos/LAB5-04_ex2_tim6.mp4 (GitHub에서 열기 ↗)



예제 ③ PB4(D5) 파형 · 2 V/칸, 200 µs/칸, 주기 1 ms · 듀티 50 %

영상: LAB5/evidence/videos/LAB5-04_ex3_pwm50_0.mp4 (GitHub에서 열기 ↗)

처음 측정에서는 5 V/칸으로 놓아 측정값 옆에 신뢰도 경고(?)가 떴고, 보정되지 않은 프로브 때문에 모서리가 튀었다. 정상 프로브로 다시 잰 화면을 사용했다.

5. 실험 5 · STM32_1 과제3 — STM32_0 과제1을 ISR로 재현

① 목적

버튼을 누를 때마다 LED 반전 간격을 1 → 2 → 4 → 8 → 1 s로 바꾸는 동작을 버튼 인터럽트와 TIM6 인터럽트만으로 구현한다.

② 동작 원리

버튼 ISR은 간격만 바꾸고, TIM6 ISR(100 ms)은 틱을 세어 간격이 차면 LED를 뒤집는다. 두 ISR이 함께 쓰는 `interval_ticks`(10 · 20 · 40 · 80)는 파일 범위의 `volatile` 변수이다. 비교는 `>=`로 해서, 8 s 중간에 1 s로 줄여도 다음 틱에 바로 반전한다.

③ 핵심 코드

```
/* STM32_1_과제3: 두 ISR이 함께 쓰는 LED 간격 (100 ms 틱 단위, 10 = 1초).
 * 버튼 ISR이 바꾸고 TIM6 ISR이 읽으므로 함수 밖에 두고 volatile을 붙인다. */
static volatile uint32_t interval_ticks = 10U;

static void button_isr_p7(void)
{
    static uint32_t last_ms = 0;
    uint32_t now_ms = HAL_GetTick();
    if (now_ms - last_ms >= 200U)          /* 200 ms 안의 반복 호출(접점 떨림)은 무시 */
    {
        last_ms = now_ms;
        /* 1초 -> 2초 -> 4초 -> 8초 -> 1초 */
        interval_ticks = (interval_ticks >= 80U) ? 10U : interval_ticks * 2U;
    }
}

static void tim6_isr_p4(void)
{
    static uint32_t count = 0;             /* 100 ms 틱 수. 이 함수만 사용 */
    if (++count >= interval_ticks)         /* >= : 간격이 갑자기 줄어도 다음 틱에 반전 */
    {
        count = 0;
        HAL_GPIO_TogglePin(LED_ece2_GPIO_Port, LED_ece2_Pin);
```

```
    }
}
```

④ 모의 실행 (실험 전)

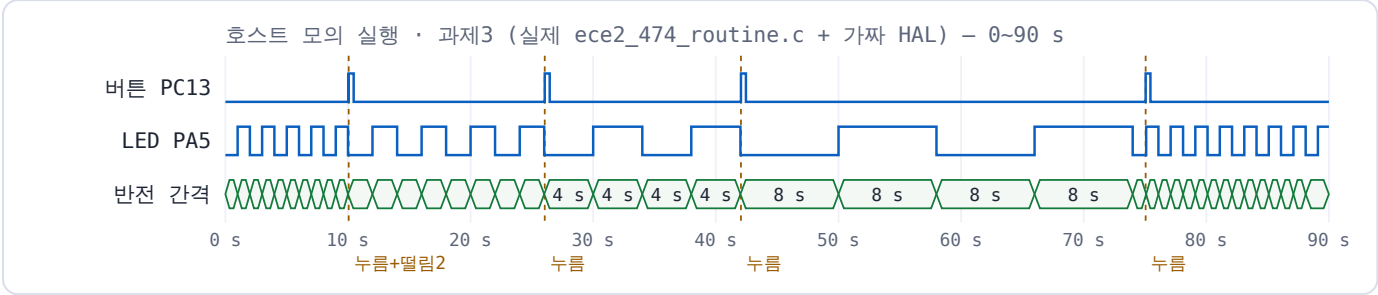
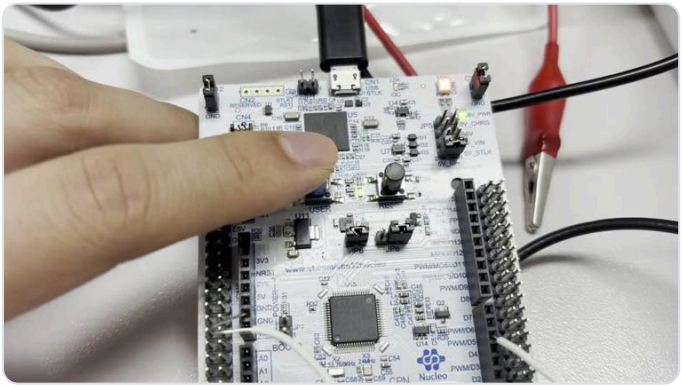


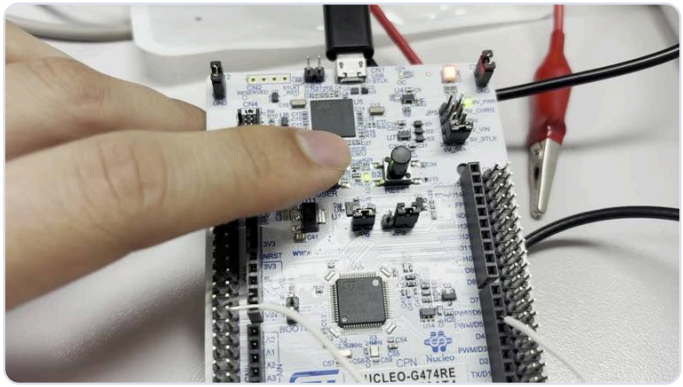
그림 5-1. 저장소 코드를 PC에서 가짜 HAL로 실행한 결과. 10.05 s의 누름과 3 ms · 7 ms 뒤 떨림 2회는 한 번으로 인정되었고, 간격은 1 → 2 → 4 → 8 → 1 s로 바뀌었다.

⑤ 보드 검증

확인 항목	기대	결과
시작	LD2 1 s 간격	일치
버튼 1 · 2 · 3 · 4번	2 · 4 · 8 · 1 s	일치
짧게 연타	한 번 누를 때 한 단계	일치
조교 확인	—	✔



과제3 시연 · 버튼으로 간격 변경



과제3 시연 · LD2 반전 (간격은 영상에서 확인)

영상: LAB5/evidence/videos/LAB5-05_task3.mp4 (GitHub에서 열기 ↗)

6. 실험 6 · STM32_1 과제4 — 버튼으로 PWM 비율 변경 · C 디버거

① 목적

버튼마다 PB4의 1 kHz PWM 비율을 25 → 50 → 75 → 25 %로 바꾸고, VS Code 디버거로 버튼 ISR 안의 변수와 비교값이 바뀌는 과정을 확인한다.

② 동작 원리

$$CCR1 = \text{pwm_percent} \times 10 \in \{250, 500, 750\}, \qquad t_{HIGH} = 0.25,\ 0.50,\ 0.75\ \text{ms}$$

HAL이 CCR 프리로드를 켜 두므로 새 비교값은 다음 PWM 주기부터 적용된다 [2].

③ 핵심 코드

```
/* STM32_1_과제4: 현재 PWM 비율(%). 디버거 Watch에서 확인할 변수.
 * TIM3 한 주기 = 1000칸(ARR 999)이므로 비교값 = 비율 x 10. */
static volatile uint32_t pwm_percent = 25U;

static void button_isr_p7(void)
{
    static uint32_t last_ms = 0;
    uint32_t now_ms = HAL_GetTick();
    if (now_ms - last_ms >= 200U)          /* 200 ms 안의 반복 호출(접점 떨림)은 무시 */
    {
        last_ms = now_ms;
        /* 25 % -> 50 % -> 75 % -> 25 % */
        pwm_percent = (pwm_percent >= 75U) ? 25U : pwm_percent + 25U;
        __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, pwm_percent * 10U);   /* 중단점(F9) 위치 */
    }
}
```

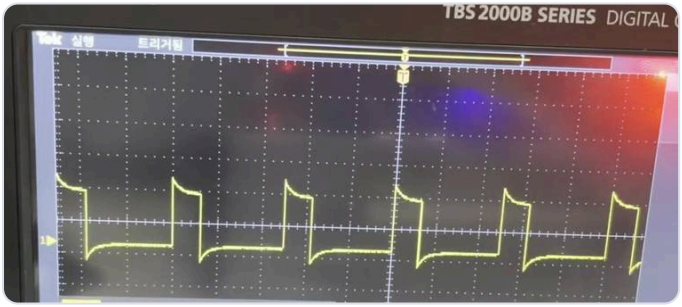
④ 모의 실행 (실험 전)



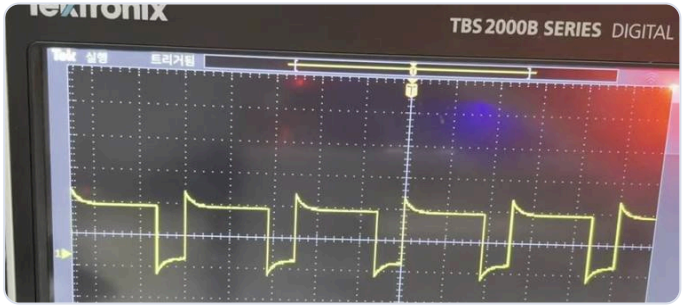
그림 6-1. 모의 실행. 시작 25 %(250)에서 버튼마다 50 → 75 → 25 → 50 %, 떨림은 무시되었다.

⑤ 보드 검증 · PWM

버튼	기대 HIGH 폭	오실로스코프 (화면 눈금으로 읽음)	결과
시작	0.25 ms (25 %)	HIGH가 주기의 약 1/4	일치
1번	0.50 ms (50 %)	HIGH와 LOW 길이 같음	일치
2번	0.75 ms (75 %)	HIGH가 주기의 약 3/4	일치
3번	0.25 ms (25 %)	처음과 같음	일치

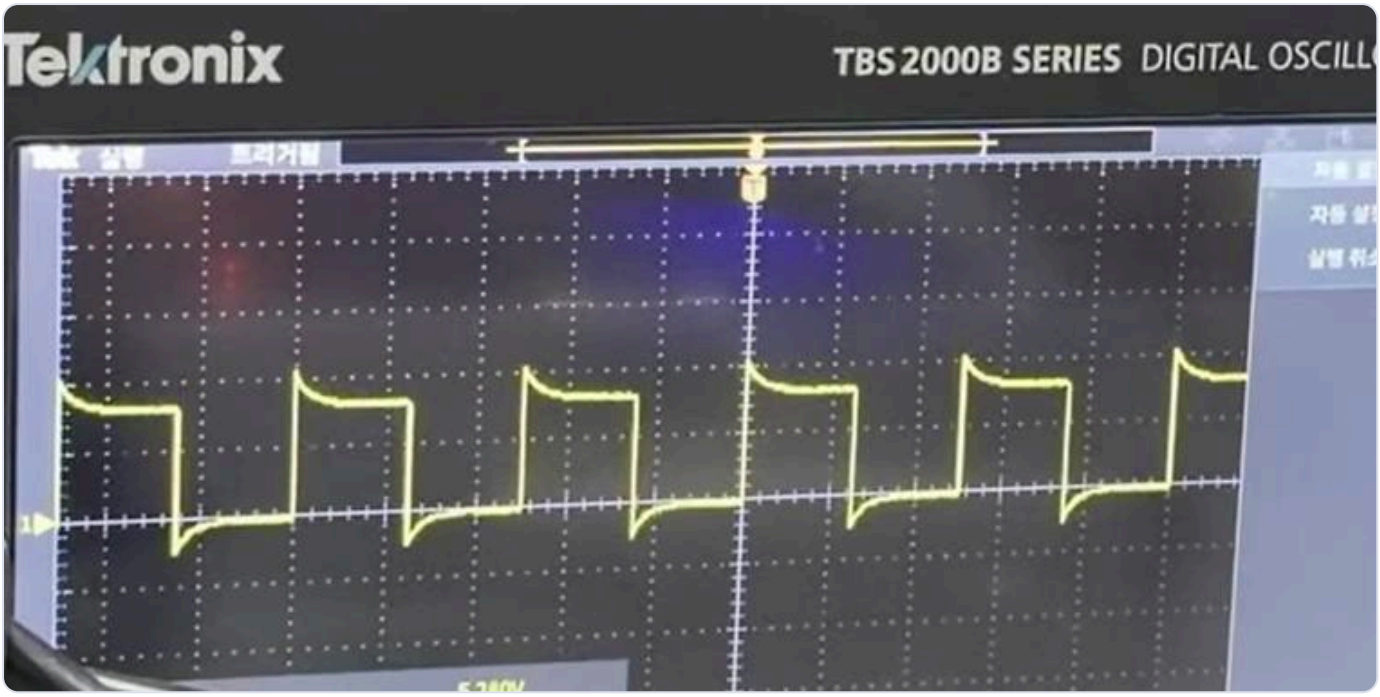


과제4 · 25 % (HIGH가 짧음)



과제4 · 75 % (HIGH가 김)

영상: LAB5/evidence/videos/LAB5-06_task4_pwm_2.mp4 (GitHub에서 열기 ↗)



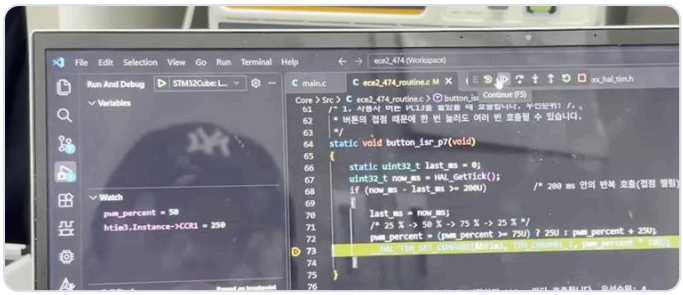
과제4 · 50 %

영상: LAB5/evidence/videos/LAB5-06_task4_pwm_1.mp4 (GitHub에서 열기 ↗)

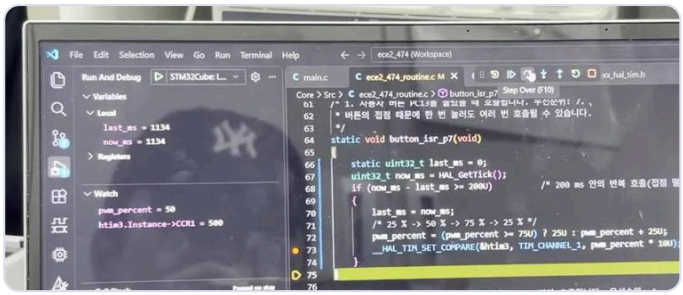
과제4 파형은 보정되지 않은 프로브로 찍어 모서리가 튀어 보인다. 이 왜곡은 HIGH 구간의 길이(비율)를 읽는 데에는 영향이 없다.

⑥ 디버거 사용 (조교 시연)

단계	조작	관찰 (Watch)	
1	73번 줄(`__HAL_TIM_SET_COMPARE`)에 중단점, 디버그 시작 → main에서 계속	25 %, 250	
2	버튼 → 73번 줄에서 정지	**pwm_percent = 50**, CCR1 = 250	비율 변수만 먼저 바뀜
3	F10(단계 건너뛰기)	pwm_percent = 50, **CCR1 = 500**	비교값 갱신
4	계속 → 버튼	75, 500 → F10 → 75, **750**	
5	중단점 해제 → 계속 → 정지	25 → 50 → 75 → 25 % 순환	



73번 줄 정지 · pwm_percent 50, CCR1 250



F10 후 · CCR1 500

영상: LAB5/evidence/videos/LAB5-06_task4_debug.mp4 (GitHub에서 열기 ↗)

디버깅 중 두 가지 현상을 겪었다. 첫째, 68번 줄에서 F10을 누르면 한 줄 실행 대신 계속 실행되어 버리는 일이 있어, 중단점을 값이 바뀌는 줄로 옮겨 진행했다. 둘째, 정지한 동안 버튼을 다시 누르면 값이 바뀌지 않았다. 정지 중에는 SysTick도 멈춰 `HAL_GetTick()`이 흐르지 않는다. 그래서 대기하던 버튼 인터럽트가 이어서 실행되면 200 ms 떨림 검사에 걸려 무시된다(고찰 3).

7. 고찰

1. **교재 핀표는 신호 이름 단위로 대조해야 한다.** 피아노와 신호등은 교재 표와 그대로 맞았다. ADC는 포트표와 하드웨어 이름 열이 서로 맞지 않았고, 메인 클록도 N8로 잘못 적혀 있었다. 칩 기준 이름(SPI_DIN = 칩의 입력)으로 읽고 나서야 동작했다. 새 장치는 표 하나만 믿지 말고 신호의 방향(누가 내보내고 누가 읽는지)까지 확인해야 한다.
2. **나눗셈 한 줄의 비용.** 피아노는 LUT 646개로 신호등(32)이나 ADC(29)보다 훨씬 크다. `CLK_HZ/(2*frequency_hz)`가 32비트 나눗셈 회로로 합성되기 때문이다. 음이 8개뿐이므로 반주기를 상수 표로 두면 크게 줄일 수 있다.
3. **디버거는 시간을 멈춘다.** 중단점에서 멈추면 CPU뿐 아니라 시간 기준(SysTick)도 멈춘다. 시간 비교로 떨림을 거르는 코드는 멈춘 동안 들어온 입력을 "너무 빠른 반복"으로 판단해 무시한다. 정상 동작의 시간 측정은 중단점을 해제하고 해야 한다는 매뉴얼 주의 사항의 이유이다. 타이머 · PWM 같은 주변장치는 멈춘 동안에도 계속 동작한다.
4. **시뮬레이션 모델이 실제 장치를 얼마나 닮았는가.** 수업 ADC TB는 디지털 응답만 흉내 내어 수업 설계를 통과시켰다. 하지만 데이터시트의 에지 규칙(DIN은 하강 에지에서 읽음, CS 하강에서 앞 0 출력, t_{QUIET})을 넣은 모델로 보면, 같은 설계에서 동시 변경 38회 · CS 40 ns · 1비트 정렬 차이가 드러났다. 외부 장치를 다룰 때는 TB의 장치 모델을 데이터시트 기준으로 만들어야 시뮬레이션이 의미가 있다.
5. **오실로스코프 측정 조건.** 3.3 V 신호를 5 V/칸으로 보면 측정 정확도 경고가 뜨고, 보정되지 않은 ×10 프로브는 사각파 모서리를 왜곡한다. 세로 크기를 신호에 맞추고 정상 프로브를 쓰자 998.4 Hz(설계값과 0.2 % 차이)와 0~3.4 V의 깨끗한 사각파를 얻었다.

참고문헌

1. [1] D. M. Harris and S. L. Harris, *Digital Design and Computer Architecture*, 2nd ed. Waltham, MA, USA: Morgan Kaufmann, 2013. ISBN 978-0-12-394424-5. — Ch. 3 순서 회로, Ch. 5 카운터, Ch. 8 I/O의 SPI · PWM · 인터럽트.
2. [2] STMicroelectronics, *stm32g4xx-hal-driver*, GitHub, https://github.com/STMicroelectronics/stm32g4xx-hal-driver, 태그 v1.2.7 (스타 53). — `HAL_TIM_PWM_ConfigChannel`의 OC1PE 프리로드, `__HAL_TIM_SET_COMPARE`.
3. [3] Analog Devices, *AD7908/AD7918/AD7928 Data Sheet*, Rev. E, analog.com (확인일 2026-10-19). — 직렬 인터페이스(DIN은 SCLK 하강 에지에서 입력, CS 하강에서 앞 0 출력, 출력 형식), 제어 레지스터, t_{QUIET} 50 ns.

수업 자료(인용 출처 수에 넣지 않음): Glaysia/ece2_474 v1.1.0의 docs/STM32_1.md · docs/FPGA_additional.md, 실습 교재 「Combo II DLD 장비를 이용한 디지털 논리 회로 설계」 2.6절 p.396 · 3.4절 · 3.5절 p.657~658.